

UAPV/CERI - BASES DE LA PROGRAMMATION L1 S1 INFORMATIQUE
Feuille de TD n° 1

Dans cette UE nous programmons en python.

Pour afficher « Bonjour » on écrit : `print(" Bonjour")`

Pour afficher le contenu d'une variable A, on écrit : `print(A)`;

On peut aussi faire `print("La valeur contenue dans la variable A est », A )`

Pour lire, on utilisera l'instruction : `a=input("Entrez une valeur: »)`

EXERCICE 1

Écrire un programme qui demande à l'utilisateur 2 entiers qu'il met dans deux variables a et b et qui affiche « plus grand » si a est supérieur à b, sinon il affiche « plus petit ou égal ».

Modifier le programme pour qu'il affiche : « plus grand », « plus petit », ou « égal ».

EXERCICE 2

Écrire un programme qui demande à l'utilisateur une longueur L et une largeur l (la longueur doit être supérieure ou égale à la largeur et $L \geq l$) et qui affiche la surface du rectangle correspondant. Si la condition sur les longueurs n'est pas respectée le programme affiche le message : « Données incohérentes »

EXERCICE 3

Écrire un programme qui lit 6 entiers et qui les met dans 6 variables : a, b, c, d, e, f. Ensuite, si au moins 3 variables sont non nulles le programme doit afficher « la saisie est correcte », sinon il affiche « la saisie est incorrecte ».

EXERCICE 4

Écrire un programme qui lit 3 nombres qu'il met dans trois variables a, b et c et qui les réordonne de façon à ce que le contenu de a soit inférieur ou égal à celui de b et celui de b soit inférieur ou égal à celui de c. Le programme affiche a, b et c pour vérification.

EXERCICE 5

Écrire un programme qui permet de saisir 10 valeurs et d'afficher la plus petite valeur, la plus grande valeur et la moyenne. Ce programme ne doit utiliser que 4 variables.

EXERCICE 6

Écrire un programme qui saisit les coordonnées (entiers) de deux points et qui affiche la distance (double) les séparant.

EXERCICE 7

Écrire un programme qui saisit les coordonnées (entiers) de 3 points et qui dit si les trois points forment un triangle rectangle.

EXERCICE 8

Écrire un programme qui demande à l'utilisateur une opération (*, +, -, /), suivie de deux opérandes. Le programme doit afficher ensuite le résultat de l'opération appliquées aux deux opérandes. Les opérandes et les résultats doivent être des entiers.

Attention : pensez à interdire la division par 0.0

Exécution :

Entrez l'opération : *

Entrez la première opérande : 3 Entrez la deuxième opérande : 4 Le résultat de 3*4 est 12

EXERCICE 9

Écrire un programme qui demande 3 valeurs a, b et c de type double et affiche les solutions de l'équation $ax^2+bx+c=0$. On vérifiera que a est différent de 0.

EXERCICE 10

Écrire un programme qui reçoit un nombre à 4 chiffres et qui affiche les chiffres un à un avec un espace les séparant. Le programme doit vérifier d'abord que le nombre contient bien 4 chiffres. Dans le cas contraire le programme affiche un message : « le nombre ne contient pas 4 chiffres »

BASES DE LA PROGRAMMATION L1 S1 INFORMATIQUE

Feuille de TD n° 2

Les boucles

EXERCICE 1: $\exists \in \mathbb{N}$ avec $m \in]0; 100[$.

Écrire un programme qui demande à l'utilisateur de lui fournir un nombre entier positif et inférieur à 100 et ceci jusqu'à ce que la réponse soit satisfaisante. Le dialogue avec l'utilisateur se présentera ainsi: donnez un entier positif inférieur à 100 : 453

donnez un entier positif inférieur à 100 : 25

merci pour le nombre 25

$\left\{ \begin{array}{l} \text{while } \text{not } (0 < m \text{ and } m < 100) \\ \text{not } (0 < m) \text{ or } 100 \leq m \\ m \leq 0 \text{ or } 100 \leq m \end{array} \right.$

EXERCICE 2: $\exists \in \mathbb{N}$ avec $m \in [2; 9]$.

Écrire un programme qui demande à l'utilisateur un nombre entre 2 et 9 et affiche la table de multiplication de celui-ci. La saisie du nombre est bloquante : tant que le nombre n'est pas compris entre 2 et 9 le programme redemande...

EXERCICE 3 :

A- Écrire un programme qui saisit une suite de valeurs entières non nulles. A la fin de la saisie, le programme doit afficher la moyenne des valeurs saisies. La saisie s'arrête quand l'utilisateur donne la valeur 0.

B- Écrire un programme qui saisit une suite de valeurs entières non nulles. A la fin de la saisie, le programme doit afficher la moyenne des valeurs saisies, la valeur minimale et la valeur maximale. Les trois valeurs doivent être calculées en utilisant une seule boucle. La saisie s'arrête quand l'utilisateur donne la valeur 0.

EXERCICE 4: 3 listes + 1 while $m \neq 0$

Écrire un programme qui saisit une suite de valeurs entières non nulles. A la fin de la saisie, le programme doit afficher la moyenne des valeurs paires et la moyenne des valeurs impaires. La saisie s'arrête quand l'utilisateur donne la valeur 0. Attention la division par zéro est une opération interdite.

EXERCICE 5

A- Écrire un programme qui permet de déterminer le juste prix (juste prix = 42). L'utilisateur saisit un nombre entier. Le programme répond : « inférieur » si le prix indiqué est supérieur à 42, il répond :

« supérieur » si le prix indiqué est inférieur à 42 et il répond : « BRAVO! » s'il trouve le prix exact. Tant que le prix n'est pas le bon, l'utilisateur doit retenter sa chance en saisissant un nouveau nombre.

B- Modifiez le programme précédent afin que le juste prix soit généré aléatoirement et pas codé en dur à 42, qui sera évidemment inconnu de l'utilisateur.

En plus le programme doit, à la fin de chaque partie, afficher le le nombre de coups de la partie et proposer à l'utilisateur s'il veut continuer le jeu. Si la réponse est « o » alors le programme lance une nouvelle partie, sinon le programme affiche le nombre de coups moyen global et affiche « A bientôt » avant d'arrêter.

EXERCICE 6 : *import turtle*

Écrire un programme qui permet de dessiner un rectangle d'étoiles *. Le programme a besoin de 2 entrées : la largeur et la hauteur du rectangle (que l'utilisateur doit fournir).

EXERCICE 7 : *import turtle*

Écrire un programme qui permet de dessiner X avec des *. Le programme a besoin d'une entrée : la taille de la forme. La taille doit être un nombre entier impair compris entre 15 et 20 (inclus). Tant que le nombre saisi ne vérifie pas la condition, le programme redemande la valeur de la taille.

EXERCICE 8

Écrire un programme qui calcule et affiche la factorielle d'un entier $n > 0$: $n! = n*(n-1)*(n-2)*...*1$; par définition $0! = 1$.

EXERCICE 9

Un nombre est dit premier s'il n'est divisible que par 1 par et lui même. Écrire un programme qui saisit un entier est qui affiche « PREMIER » s'il est premier et « PAS PREMIER » sinon. Modifier le programme pour qu'il saisisse deux entiers positifs et qu'il affiche tous les entiers premiers entre les deux.

TD3: les fonctions

L1 Math-info, 2025-2026

Dans cette série nous utiliserons seulement while pour exprimer les boucles

1- Définissez une fonction `surfCercle(R)`. Cette fonction doit renvoyer la surface (l'aire) d'un cercle dont on lui a fourni le rayon R en argument. Par exemple, l'exécution de l'instruction : `print(surfCercle(2.5))` doit donner le résultat 19.635

2- Définissez une fonction `maximum(n1,n2,n3,n4)` qui renvoie le plus grand des 4 nombres n1, n2, n3 et n4 fournis en arguments. Par exemple, l'exécution de l'instruction : `print(maximum(1,5,4,6))` doit donner

La valeur maximale est : 6.

3- Nous souhaitons résoudre l'équation : $ax^2+bx+c=0$ pour tous a, b et c.

Si $a == 0$

 si $b == 0$

 si $c == 0$

$S = \mathbb{R}$

 else

 il n'y a pas de solution

 else

$S = \{-c/b\}$

else

$\Delta = \dots$

 si $\Delta == 0$

$S = \{-b/2a\}$

 sinon si $\Delta > 0$

 sinon

3.1 Ecrire la fonction qui calcule le discriminant (Delta)

3.2 Ecrire la fonction, **resoudre**, qui résout l'équation. Attention la fonction ne doit faire aucun affichage.

3.3 Ecrire une fonction d'affichage qui permet après avoir utilisé la fonction résoudre, d'afficher le résultat de la résolution. Voici un exemple:

Entrez a, b et c :

0 2 1

Votre équation admet une seule solution : -0.5

4.1 Une action est cotée à 57€. Sa valeur augmente de 3% tous les mois. Écrire une fonction en Python qui renvoie le nombre de mois à attendre pour que sa valeur dépasse 200€.

4.2 Proposer une généralisation de la fonction précédente : étant donnée le prix initiale d'une action, étant donné le taux d'augmentation de l'action à chaque mois (dans 4.1 le taux est 0.03), la fonction doit renvoyer le nombre de mois pour atteindre une valeur donnée.

5- Ecrire une fonction « somme » qui prend en argument un entier positif et qui renvoie la somme de tous les chiffres le composant. Par exemple, « `print(somme(1234))` » affichera 10

6- Ecrire une fonction Python bissextile prenant en entrée un entier représentant une année et retournant un booléen indiquant si l'année est bissextile. On rappelle qu'une année est bissextile si elle possède un 29 février. Une année est bissextile si elle est multiple de 4 mais pas multiple de 100, ou si elle est multiple de 400.

Ex : bissextile(2024) doit renvoyer True, bissextile(1800) doit renvoyer False et bissextile(2000) doit renvoyer True

7-

A- Écrire une fonction qui détermine si deux nombres réels sont identiques à un epsilon près. Exemple : 2.01 et 2.005 sont identiques à 0.01 près, car $|2.01 - 2.005| = 0.005 < 0.01$.

On considère que deux nombres réel sont identique si ils le sont à $1e-6$.

B- Écrire une fonction qui prend en argument 3 nombres réels et qui retourne True/False selon qu'ils peuvent représenter les carrés des longueurs d'un triangle rectangle.

C- Écrire une fonction qui prend en argument 4 nombres réels et qui retourne True/False si trois d'entre eux peuvent représenter les carrés les longueurs d'un triangle rectangle.

8-

Voici la méthode de décomposition d'un entier en facteurs premiers:

Entrée : un entier naturel $n > 1$

Sortie : une chaîne de caractères qui représente la décomposition de n en facteurs premiers

1. Initialiser une chaîne `resultat` vide.
2. Tant que $n > 1$, faire :
 - a. Trouver le plus petit diviseur premier d de n .
 - b. Si `resultat` est vide, mettre d dans `resultat`.
 - c. Sinon, ajouter " x " puis d à la fin de `resultat`.
 - d. Remplacer n par n/d (division entière).
3. Quand $n = 1$, arrêter.

A- Ecrire une fonction **est_premier(n)** qui retourne True si n est un nombre premier, sinon False.

B- Ecrire une fonction **plus_petit_diviseur(n)** qui utilise `est_premier` et retourne le plus petit diviseur premier de n (supérieur à 1).

C- Ecrire une fonction **decomposition_facteurs(n)** qui utilise plusieurs appels à `plus_petit_diviseur` pour produire la décomposition en facteurs premiers de n .

Ici exceptionnellement on va utiliser une variable de type **str** (chaine). On début on fait : **resultat=""**. Imaginons que **resultat** vaut à un moment "2x2" et qu'on veut ajouter **d** (un entier) comme facteur, alors on fait simplement : **resultat=resultat+str(d)**

TD5 : listes et fonctions

Dans tous les exercices seules les boucles avec while sont autorisées

Partie I.

Pour un entier donné, on peut montrer que s'il n'existe aucun diviseur strictement inférieur à sa racine carrée, alors il n'en existe pas non plus au-dessus de cette racine carrée.

- 1- En utilisant la propriété ci-dessus, écrire une fonction qui dit si un entier est premier
- 2- Ecrire une fonction qui renvoie la liste de tous les nombres premiers inférieurs ou égaux à un entier n donné.

Le crible d'Ératosthène

Principe : on élimine progressivement les multiples des nombres premiers, en partant du plus petit, pour ne conserver que les nombres premiers.

Étapes de l'algorithme (exemple jusqu'à n) :

1. On écrit tous les entiers de 2 à n .
 2. On commence avec $p = 2$, le premier nombre premier.
 3. On barre tous les **multiples de p** (sauf lui-même).
 4. On passe au prochain nombre non barré.
 5. On répète l'opération jusqu'à $\sqrt{n}$.
 6. Les nombres non barrés restants sont les **nombres premiers**.
- 3- Ecrire une fonction qui prend en argument un entier n et qui renvoie la liste de tous les nombres premiers qui lui sont inférieurs (ou égal).

Pour simuler « On barre tous les **multiples de p** », on utilise une liste supplémentaire contenant des valeurs « False ». Chaque fois qu'on barre un élément le « False » passe à True.

Afin d'accéder directement au multiple de p , utiliser des liste qui stocke les valeur à partir de 0 (sachant que 0 et 1 ne sont pas premiers)

Partie II.

1. Ecrire une fonction qui prend en argument deux listes triées par ordre croissant et qui renvoie une liste triée contenant tous les entiers des deux listes.

2-

Principe du tri par insertion

1. On parcourt la de gauche à droite.
2. À chaque étape, on **prend l'élément courant** et on l'insère **à sa place dans la partie déjà triée** à gauche.
3. La partie gauche est toujours triée, et on "glisse" les éléments plus grands vers la droite pour faire de la place.

Étapes sur [5, 2, 9, 1, 5, 6]

1. Commence avec 2 : [5, 2, 9, 1, 5, 6] → insert 2 avant 5
→ [2, 5, 9, 1, 5, 6]
2. Ensuite 9 : déjà à la bonne place → [2, 5, 9, 1, 5, 6]
3. Ensuite 1 : on décale 9, 5, 2 → [1, 2, 5, 9, 5, 6]
4. Puis 5 : insérer avant 9 → [1, 2, 5, 5, 9, 6]
5. Enfin 6 : insérer avant 9 → [1, 2, 5, 5, 6, 9]

Ecrire une fonction qui prend en argument une liste et qui la trie en utilisant l'algorithme ci-dessus. Dans cet exercice aucun élément ne doit être supprimé, ni ajouté. Seuls des échanges de valeurs sont autorisés.

3- Utiliser les deux fonctions ci-dessus pour trier une liste (deux tris + une fusion).

Contrôle des connaissances de programmation en python

L1 informatique-Mathématique

Durée : 1h20

Exercice 1 : pgcd et ppcm

1.1 Écrire une fonction qui prend en argument une liste L d'entiers et un entier x, et qui vérifie si x appartient à L. Elle renvoie vrai si x est dans L, et faux sinon.

1.2 Écrire une fonction qui reçoit une liste en argument et qui renvoie une nouvelle liste ne contenant aucun doublon. Utiliser la fonction de la question 1.1.

Exemple : `sans_doublons([1, 3, 7, 3, 9, 1, 1, 10])` → `[1, 3, 7, 9, 10]`

1.3 Écrire une fonction qui reçoit deux listes **sans doublons** et qui renvoie une liste contenant les entiers appartenant aux deux listes en même temps (intersection). Utiliser la fonction de la question 1.1.

Exemple :

`L1 = [1, 3, 7, 9, 4] ; L2 = [3, 9, 11, 6, 7]`

`inter(L1, L2)` → `[3, 7, 9]`

1.4 Écrire une fonction qui reçoit un entier n et qui renvoie une liste contenant tous ses diviseurs.

1.5 Écrire une fonction qui reçoit une liste d'entiers en argument et qui renvoie le plus grand élément.

1.6 Le PGCD (plus grand diviseur commun) de deux entiers est le plus grand entier qui divise les deux. Écrire une fonction `pgcd` en utilisant les fonctions des questions 1.3, 1.4 et 1.5.

1.7 Écrire une fonction qui reçoit deux entiers n et m et qui renvoie une liste contenant tous les multiples de n jusqu'à la valeur m.

1.8 Écrire une fonction qui reçoit une liste d'entiers en argument et qui renvoie le plus petit élément.

1.9 Le PPCM (le plus petit multiple commun) de deux entiers n et p est le plus petit entier qui est multiple de n et de p. En utilisant les fonctions des questions 1.3, 1.7 et 1.8, écrire la fonction `PPCM`. (Indication : $m=n \times p$ et un multiple de n et de p)

Exercice 2 : Fusion de deux listes triées avec contraintes

On considère deux listes d'entiers **triées dans l'ordre croissant** : L1 et L2.

On souhaite fusionner ces deux listes en une seule, **toujours triée**, mais **avec les contraintes suivantes** :

1. **Un seul parcours** des listes :

- Vous n'avez pas le droit de réutiliser `sorted()` ou toute autre méthode de tri.
- Vous ne pouvez pas parcourir plusieurs fois L1 ou L2.
- Vous devez avancer progressivement dans les deux listes.

2. **Aucune nouvelle liste créée** :

- La fusion doit se faire **en modifiant l'une des deux listes existantes** (au choix),
- L'utilisation d'une troisième liste intermédiaire est interdite.

Vous pouvez utiliser : `append` (ajoute à la fin), `insert` (insère à un indice donné) et `len` (calcule la taille).

Rappel : `L1.insert(i, x)` permet d'insérer `x` dans `L` à l'indice `i` en décalant tous les éléments après `i` (`i` inclu) vers la droite.

Exemple:

`L1 = [1, 4, 6, 10]`

`L2 = [2, 3, 5, 7, 8]`

`fusion_en_place(L1, L2)`

`print(L1)` # Résultat : `[1, 2, 3, 4, 5, 6, 7, 8, 10]`